

STA 5635 HW 9

Palmer Swanson

March 12, 2026

Problem 1

x_min	y_min	Expresssion 1	Expression 2	Expression 3
0.6828	0.5218	0.1722	0.1722	0.1722

Table 1: Minimized x and y values for the three expressions, as well as the values of the expressions.

Problem 2

Part a

Viterbi sequence:

[illegible]

Part b

The ratio of alpha values at time 118: 0.4970

The ratio of beta values at time 118: 1.3175

Problem 3

$$\hat{\pi} = \begin{bmatrix} 1.0000 & 0.0000 \end{bmatrix}$$

$$\hat{A} = \begin{bmatrix} 0.8698 & 0.1302 \\ 0.2087 & 0.7913 \end{bmatrix}$$

$$\hat{B} = \begin{bmatrix} 0.2128 & 0.2143 & 0.2046 & 0.1896 & 0.0896 & 0.0891 \\ 0.1061 & 0.0921 & 0.1148 & 0.2087 & 0.1861 & 0.2921 \end{bmatrix}$$

Code

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.optimize import minimize
5
6 #question 1.
7 sigma = 0.5
8
9 #define the functions
10 def a(x):
11     temp = x - 1 - np.log(x)
12     return temp
13
14 def b(x,y):
15     temp = (1+y-x)**2/(1+(2*y))
16     return temp
17
18 #"as close to each other as possible". Objective function?
19 def objective(vars):
20     x, y = vars
21
22     variable1 = a((x*(sigma**2 + 1))/(2*sigma**2))
23     variable2 = b(x, y)/2.0
24     variable3 = a(y)
25
26     #Euclidean distance between the three variables. Want to minimize this.
27     #pdist wasn't working, so I just wrote it out.
28     temp = ((variable1-variable2)**2 +
29             (variable1-variable3)**2 +
30             (variable2-variable3)**2)
31
32     return temp
33
34 #minimize this all. Need an itial guess. Let's say x = 1, y = 1
35 #need bounds though, because log(x) is undefined for x <= 0. denominator in
    b(x,y) just can't be -0.5, which is covered if
36 #we bound both x and y > 0. bounds give a minimum and no maximum.
37 initial_guess = [1, 1]
38 bounds = [(1e-8, None), (1e-8, None)]
39 result = minimize(objective, initial_guess, bounds=bounds)
40
41 #get the result. Also want to see the value of the functions at these minimized
    values
42 x_min, y_min = result.x
43 v1 = a((x_min*(sigma**2 + 1))/(2*sigma**2))
```

```

44 v2 = b(x_min, y_min)/2.0
45 v3 = a(y_min)
46
47 #put this into a dataframe for the writeup, rounded to 4 decimal places.
48 df1 = pd.DataFrame({
49     "x_min": [x_min],
50     "y_min": [y_min],
51     "Expression_1": [v1],
52     "Expression_2": [v2],
53     "Expression_3": [v3]
54 }).round(4)
55
56 #need to send this table to LATEX
57 LATEX_table = df1.to_latex(
58     index=False,
59     caption=None,
60     label=None,
61     column_format="c" * df1.shape[1],
62     escape=True,
63     float_format="%.4f"
64 )
65
66 #save latex table
67 with open("assignments/hw09/output/smallest_test_error.tex", "w") as f:
68     f.write(LATEX_table)
69
70 #plotting this. Not needed, but a nice visual
71 x_vals = np.linspace(0.01,5,500)
72 y_vals = np.linspace(0.01,5,500)
73 X, Y = np.meshgrid(x_vals, y_vals)
74
75 vals = [X,Y]
76 Z = objective(vals)
77
78 plt.figure(figsize=(6,5))
79 plt.contourf(X,Y,Z,levels=50)
80 plt.scatter(x_min,y_min,color='black',s=80)
81 plt.colorbar(label="Distance between the three expressions")
82
83 plt.xlabel("x")
84 plt.ylabel("y")
85 plt.title("Contour plot of distance between the three expressions")
86 plt.savefig("assignments/hw09/figures/contour_of_solution.pdf", dpi=400,
87     bbox_inches="tight")
88
89

```

```

90 #problem 2, sequence of dice rolls x from a dishonest casino model.
91 #states are fair and loaded.
92 #https://en.wikipedia.org/wiki/Viterbi_algorithm#Pseudocode
93 df_q2 = pd.read_csv("assignments/hw09/data/hmm_pb1.csv", header=None)
94
95 #part a, Viterbi algo.
96 #states, 1 = Fair, 2 = Loaded
97 states = [1, 2]
98
99 #initial probabilities
100 pi = np.array([0.5, 0.5])
101
102 #chose these probabilities from
103 #https://hmmlearn.readthedocs.io/en/latest/auto_examples/plot_casino.html
104 trans = np.array([
105     [0.95, 0.05], #fair
106     [0.10, 0.90] #loaded
107 ])
108
109 #need emission probabilities. P(roll | state)
110 #fair die, 1/6 for each roll. loaded die, 1/10 for 1-5 and 1/2 for 6.
111 emit = {
112     1: {1: 1/6, 2: 1/6, 3: 1/6, 4: 1/6, 5: 1/6, 6: 1/6},
113     2: {1: 1/10, 2: 1/10, 3: 1/10, 4: 1/10, 5: 1/10, 6: 1/2}
114 }
115
116 #function for Viterbi algorithm
117 def Viterbi(states, init, trans, emit, obs):
118     #get number of observations
119     T = len(obs)
120     S = len(states)
121
122     #need log probabilities
123     log_init = np.log(init)
124     log_trans = np.log(trans)
125
126     #need prob and prev. need prev to be integer, bc it is storing the state
127     #init as -inf bc of log space
128     log_prob = np.full((T, S), -np.inf)
129     prev = np.zeros((T, S), dtype=int)
130
131     #initialize
132     #need to remember that python indexes start from 0, need to subtract 1 from
133     #state to get the right index in the prob array
134     for s in states:
135         log_prob[0][s-1] = log_init[s-1] + np.log(emit[s][obs[0]])

```

```

136     #recursion portion
137     for t in range(1, T):
138         for s in states:
139             for r in states:
140                 #summations bc of log space
141                 new_prob = (
142                     log_prob[t-1][r-1] + log_trans[r-1][s-1] +
143                     np.log(emit[s][obs[t]])
144                 )
145
146                 if new_prob > log_prob[t][s-1]:
147                     log_prob[t][s-1] = new_prob
148                     prev[t][s-1] = r
149
150     #empty array of length T. Need this to be integer
151     path = np.zeros(T, dtype=int)
152     path[T - 1] = np.argmax(log_prob[T - 1]) + 1
153
154     for t in range(T - 2, -1, -1):
155         path[t] = prev[t + 1][path[t+1] - 1]
156
157     return path
158
159 path = Viterbi(states=states, init=pi, trans=trans, emit=emit,
160               obs=df_q2.iloc[0].to_numpy())
161
162 #breaking this into chunks to make it easier to read
163 chunk_size = 50
164
165 with open("assignments/hw09/output/viterbi_path.tex", "w") as f:
166     f.write("\\[\n")
167     f.write("\\begin{aligned}\n")
168     f.write("&\\text{Viterbi\_sequence:}\n")
169
170     for i in range(0, len(path), chunk_size):
171         chunk = path[i:i+chunk_size]
172         f.write("& " + " ".join(str(x) for x in chunk) + "\n")
173
174     f.write("\\end{aligned}\n")
175     f.write("\\]\n")
176
177 #plot this? I see a state sequence
178 plt.figure()
179 plt.plot(path, drawstyle="steps-mid")
180 plt.yticks([1,2])
181 plt.xlabel("Time")
182 plt.ylabel("State")

```

```

181 plt.title("Most Likely Hidden State Sequence (Viterbi)")
182 plt.savefig("assignments/hw09/figures/state_sequence.pdf", dpi=400,
183             bbox_inches="tight")
184 plt.close()
185 #plt.show()
186
187 #need to do something with this. Find a better way to report
188
189 #implement the forward algorithm for the same sequence of dice rolls.
190 #need to renormalize the alpha values at each step to prevent underflow.
191 #for forward pass, store scaling factor at each time step
192 def forward_algorithm(states, init, trans, emit, obs):
193     #length of the dice rolls
194     T = len(obs)
195
196     #number of hidden states
197     S = len(states)
198
199     #initialize alpha. alpha[t,s] is the forward probability \alpha_t^s.
200     #each row is a time t and each column is a state s
201     alpha = np.zeros((T, S))
202
203     #scaling factor used at time t.
204     u = np.zeros(T)
205
206     #\alpha_1^k = \pi_k * b_{x_1}^k
207     for s in states:
208         alpha[0][s-1] = init[s-1] * emit[s][obs[0]]
209
210     #scaling alpha 0
211     u[0] = np.sum(alpha[0])
212     alpha[0] = alpha[0] / u[0]
213
214     for t in range(1, T):
215         for s in states:
216             sum_alpha = 0
217
218             #sum over all possible previous states r.
219             for r in states:
220                 sum_alpha += alpha[t-1][r-1] * trans[r-1][s-1]
221
222             #multiply by the emission prob
223             alpha[t][s-1] = sum_alpha * emit[s][obs[t]]
224
225     #scaling alpha t
226     u[t] = np.sum(alpha[t])
227     alpha[t] = alpha[t] / u[t]

```

```

227
228     #asked to report the ratio of the alpha values at time 118.
229     #time 118 -> row 118
230     #state 1 is column 0, state 2 is column 1. So we want alpha[118,0] /
231     alpha[118,1]
232     ratio = alpha[118,0] / alpha[118,1]
233
234     #ratio, alpha, and u. u is needed for backward algo
235     return alpha, ratio, u
236
237 #function to implement the backward algorithm for the same sequence of dice
238 rolls.
239 #using the algorithm from pg 23 of the slide
240 def backward_algorithm(states, trans, emit, obs, u):
241     #number of observations
242     T = len(obs)
243
244     #number of hidden states
245     S = len(states)
246
247     #store the backward probabilities. beta[t,s] is the backward probability
248     \beta_t^s.
249     beta = np.zeros((T, S))
250
251     #initialize beta. \beta_T^k = 1 for all k.
252     beta[T-1] = np.ones(S)
253
254     for t in range(T-2, -1, -1):
255         for s in states:
256             sum_beta = 0
257
258             #sum over all possible next states r.
259             for r in states:
260                 sum_beta += trans[s-1][r-1] * emit[r][obs[t+1]] * beta[t+1][r-1]
261
262             beta[t][s-1] = sum_beta/u[t+1]
263
264     #asked to report the ratio of the alpha values at time 118.
265     #time 118 -> row 118
266     #state 1 is column 0, state 2 is column 1. So we want beta[118,0] /
267     beta[118,1]
268     ratio = beta[118,0] / beta[118,1]
269     return ratio, beta
270
271 alphas, alpha_ratio, u = forward_algorithm(states=states, init=pi, trans=trans,
272     emit=emit, obs=df_q2.iloc[0].to_numpy())

```

```

269 beta_ratio, betas = backward_algorithm(states=states, trans=trans, emit=emit,
    obs=df_q2.iloc[0].to_numpy(), u=u)
270
271 print("Ratio of alpha values at time 118:", alpha_ratio)
272 print("Ratio of beta values at time 118:", beta_ratio)
273
274 #Question 3, impliment the Baum-Welch algorithm using the forward and backward
    algorithms from q2.
275 #Initialize \pi, a, b with a guess of our choice. report the final values of
    \pi, a, b.
276
277 df_q3 = pd.read_csv("assignments/hw09/data/hmm_pb2.csv", header=None)
278
279 #need to define the Baum-Welch algorithm.
280 def baum_welch(observed, seed=123, max_iters = 400, tol=1e-6):
281     #define the random generator for reproducibility
282     rng = np.random.default_rng(seed)
283
284     states = [1, 2]
285     T = len(observed)
286
287     #need to initialize parameters at random.
288     pi = rng.random(2)
289     pi = pi/pi.sum() #normalize to sum to 1
290
291     A = rng.random((2,2))
292     A = A/np.sum(A, axis=1, keepdims=True) #normalize rows to sum to 1
293
294     B = rng.random((2,6))
295     B = B/np.sum(B, axis=1, keepdims=True) #normalize rows to sum to 1
296
297     #track the log likelihood at each iteration to check for convergence.
298     #https://en.wikipedia.org/wiki/Forward%E2%80%93backward_algorithm
299     #The product of the scaling factors is the total probability for observing
    the given events irrespective of the final states:
300     log_likelihoods = []
301
302     for iter in range(max_iters):
303         if iter % 10 == 0:
304             print(f"Iteration {iter}")
305
306         #get emissions for current params instead of a static emission mat
307         emit_current = {
308             1: {1: B[0, 0], 2: B[0, 1], 3: B[0, 2], 4: B[0, 3], 5: B[0, 4], 6:
309                 B[0, 5]},
310             2: {1: B[1, 0], 2: B[1, 1], 3: B[1, 2], 4: B[1, 3], 5: B[1, 4], 6:
311                 B[1, 5]}

```



```

310     }
311     #E step: compute the forward and backward probabilities
312     #Expectation step: compute the forward and backward probabilities
313     alpha, _, u = forward_algorithm(states=states, init=pi, trans=A,
emit=emit_current, obs=observed)
314     _, beta = backward_algorithm(states=states, trans=A, emit=emit_current,
obs=observed, u=u)
315
316     #update log likelihood
317     log_likelihood = np.sum(np.log(u))
318     log_likelihoods.append(log_likelihood)
319
320     #computing gamma from the slides
321     gamma = np.zeros((T, 2))
322     for t in range(T):
323         denom = 0.0
324         for k in range(2):
325             denom += alpha[t, k] * beta[t, k]
326         for i in range(2):
327             gamma[t, i] = (alpha[t, i] * beta[t, i])/denom
328
329     #computed Xi from the slides
330     xi = np.zeros((T-1, 2, 2))
331     for t in range(T-1):
332         denom = 0.0
333         for k in range(2):
334             for l in range(2):
335                 denom = denom + alpha[t, k]*A[k, l] * beta[t+1, l]*B[l,
observed[t+1] - 1]
336
337     #kept getting errors here. Debugging stuff
338     # if denom == 0 or np.isnan(denom):
339     #     print("problem at t =", t)
340     #     print("observed[t+1] =", observed[t+1])
341     #     print("alpha[t] =", alpha[t])
342     #     print("beta[t+1] =", beta[t+1])
343     #     print("A =\n", A)
344     #     print("B =\n", B)
345
346     for i in range(2):
347         for j in range(2):
348             numer = alpha[t, i]*A[i, j] * beta[t+1, j]*B[j,
observed[t+1] - 1]
349             xi[t, i, j] = numer/denom
350
351     #M step: update the parameters
352     #update pi

```

```

353     pi_new = gamma[0]
354
355     A_new = np.zeros((2, 2))
356     for i in range(2):
357         denom = np.sum(gamma[:-1, i])
358         for j in range(2):
359             A_new[i, j] = np.sum(xi[:, i, j])/denom
360
361     B_new = np.zeros((2, 6))
362     for i in range(2):
363         denom = np.sum(gamma[:, i])
364         for k in range(1, 7):
365             numer = 0.0
366             for t in range(T):
367                 #indicator variable for if x=k
368                 if observed[t] == k:
369                     numer = numer + gamma[t, i]
370             B_new[i, k-1] = numer/denom
371
372     #update parameters
373     pi = pi_new
374     A = A_new
375     B = B_new
376
377     #convergence check. See if the log likelihood has changed by less than
the tolerance level. If it has, we can stop.
378     #don't need this for the hw
379     if len(log_likelihoods) > 1 and abs(log_likelihoods[-1] -
log_likelihoods[-2]) < tol:
380         print(f"Converged at iteration {iter}")
381
382     #plot the log likelihood trace
383     plt.figure()
384     plt.plot(log_likelihoods)
385     plt.xlabel("Iteration")
386     plt.ylabel("Log Likelihood")
387     plt.title(f"Baum-Welch Log-Likelihood Trace (tol={tol})")
388     plt.grid(True)
389     plt.savefig("assignments/hw09/figures/log_likelihood_trace.pdf",
dpi=400, bbox_inches="tight")
390     plt.close()
391     #plt.show()
392
393     break
394     return pi, A, B, log_likelihoods
395
396

```

```

397 pi_hat, A_hat, B_hat, log_likelihoods = baum_welch(df_q3.iloc[0].to_numpy(),
398           max_iters=2000, tol=1e-3)
399
400 #save these estimates to a latex file. Round to 4 decimal places for
401 readability.
402 #updates whenever this code is run
403 with open("assignments/hw09/output/hmm_estimates.tex", "w") as f:
404
405     #a whole bunch of escape characters \
406     #pi
407     f.write("\\[\n")
408     #\hat{\pi} =
409     f.write("\\hat{\pi}_=\n")
410     f.write("\\begin{bmatrix}\n")
411     #loop through \pi values and write them to the file, rounded to 4 decimal
412 places.
413     f.write("_&_".join(f"{x:.4f}" for x in pi_hat))
414     f.write("\n\\end{bmatrix}\n")
415     f.write("\\]\n\n")
416
417     #A
418     f.write("\\[\n")
419     f.write("\\hat{A}_=\n")
420     f.write("\\begin{bmatrix}\n")
421     #loop over rows
422     for row in A_hat:
423         #quad backslashes to escape
424         f.write("_&_".join(f"{x:.4f}" for x in row) + "_\\\\\n")
425     f.write("\\end{bmatrix}\n")
426     f.write("\\]\n\n")
427
428     #Emission matrix B
429     f.write("\\[\n")
430     f.write("\\hat{B}_=\n")
431     f.write("\\begin{bmatrix}\n")
432     for row in B_hat:

```